

01 · Was ist Echtzeit

Einführung und Paradigmenwechsel · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 58

Willkommen zur Vorlesung Echtzeitsysteme!

- Neun Termine: je 4 Stunden Theorie und Praxis / Übungen
- Klausur am Ende des Semesters
- Fokus: moderne Multicore-Architekturen in der Industrie
- Hardware: STM32H755 (Dual-Core Cortex-M7/M4)

Notizen

Herzlich willkommen zur Vorlesung Echtzeitsysteme. In den nächsten neun Terminen verbinden wir Theorie mit Praxis an Industrie-Hardware. Der STM32H755 dient als Dual-Core-Plattform für Echtzeit und Multicore-Synchronisation.

Folie

2 / 58

Agenda für heute

1. Organisatorisches und Spielregeln
2. Motivation und Definition von Echtzeit
3. Bare-Metal: Polling, Interrupts, Foreground/Background
4. Deep Dive: Cortex-M, NVIC und Tail-Chaining
5. Hardware STM32H755 und Toolchain
6. Laborübung am Board

Organisatorisches

- Prüfungsleistung: Klausur am Ende des Semesters (Modul Rechnersysteme)
- Materialien: Folien, Handouts, Übungen und Musterlösungen auf der Kurswebsite und in Moodle
- Labor-Hardware: Boards werden gestellt (STM32H755)
- Tools: STM32CubeIDE (C/C++)

Notizen

Alle Folien und Übungsgerüste stehen online. Die Boards bekommen Sie gestellt. Echtzeitverhalten versteht man oft erst, wenn ein Timer-Interrupt nicht rechtzeitig feuert — deshalb Praxispflicht.

Literatur und Quellen

- Hermann Kopetz: *Real-Time Systems* (Design Principles)
- Giorgio C. Buttazzo: *Hard Real-Time Computing Systems* (Scheduling)
- STMicroelectronics: Reference Manual STM32H745/755 (RM0399)
- FreeRTOS Documentation (online)
- Tipp: Datenblätter und Reference Manuals sind Ihre wichtigste Alltagsquelle

Notizen

Kopetz und Buttazzo decken Theorie und Scheduling ab. Für die Praxis brauchen Sie ständig das RM0399 — nicht auswendig, aber navigierbar: Timer, Interrupt-Controller, Speicherarchitektur.

Einstieg: Diskussion

- Stellen Sie sich vor: Ihr ABS reagiert erst 200 ms später — ist das noch „richtig“ oder schon „falsch“? (*Diskussionsannahme: 200 ms ist keine reale ABS-Spezifikation.*)

Notizen

Kurze Diskussionsrunde. Ziel: Intuition, dass funktionale und zeitliche Korrektheit zusammengehören. Die 200 ms sind eine Diskussionsannahme, keine universale Auslösefrist. Die Antwort „unter dieser Annahme falsch“ führt zur Motivation.

Motivation: Warum Echtzeit?

- Steigende Komplexität in Embedded Systems
- Autonomes Fahren, Medizintechnik, Industrie 4.0 (Beispiele, keine pauschale Klassenzuordnung)
- Desktop-OS (Windows/Linux) liefern typischerweise keine harten Fristgarantien
- Timing-Fehler können sicherheitsrelevant sein — aber nicht jede Fristverletzung ist „katastrophal“
- Ziel: vorhersagbare Reaktionen innerhalb spezifizierter Fristen

Notizen

Wenn der Browser eine halbe Sekunde hängt, ist das ärgerlich. Wenn die Fräsmaschinensteuerung hängt, wird es gefährlich. Es geht nicht um „schnell“, sondern um spezifizierte, vorhersagbare Reaktionszeiten. Katastrophale Folgen sind ein möglicher Kontext — keine allgemeine Definition von Echtzeit.

Der große Mythos der Echtzeit

Was viele denken:

- „Echtzeit = extrem schnell“
- „4 GHz ist echtzeitfähiger als 400 MHz“
- „Echtzeit = flüssiges Video“

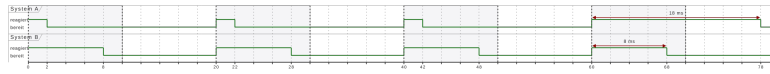
Was es wirklich ist:

- Echtzeit = Rechtzeitigkeit (Fristvertrag)
- Ein 400-MHz-Controller mit Garantie kann echtzeitfähig sein
- Ein 4-GHz-PC ohne Garantie ist es oft nicht

Notizen

Echtzeit hat nichts mit maximalem Durchsatz zu tun. Ein kleiner Mikrocontroller mit fester 10-ms-Reaktion schlägt einen Server, der im Mittel 1 ms braucht, aber gelegentlich 50 ms blockiert. Lernkontrolle: Reicht ein Mittelwert von 1 ms für eine Deadline von 10 ms? Nein — ohne Schranke/Garantie sagt der Mittelwert allein nichts über harte Fristen.

Mythos: Mittelwert $\neq$ harte Garantie



- Illustrative Einzelbeobachtungen, graue Fenster = Deadline $D = 10$ ms
- A: 2/2/2/18 ms (Mittel 6 ms), B: 8/8/8/8 ms (Mittel 8 ms)
- A ist im Mittel schneller als B — und verletzt trotzdem die Frist
- Vier Messungen bei B sind kein allgemeiner Nachweis; eine separat angenommene Schranke wäre z. B. $R \leq 9$ ms
- Rechtzeitigkeit braucht eine spezifizierte Obergrenze, nicht nur einen Mittelwert

Notizen

Didaktischer Kontrast: „schneller im Mittel“ ist keine harte Garantie. Folge B erfüllt in diesen vier Beobachtungen die Deadline, belegt aber ohne zusätzliche Schranke noch keinen allgemeinen Beweis. Die Schranke $R \leq 9$ ms ist hier eine Lehrannahme, keine Messaussage.

Konsequenzen von Timing-Fehlern

- Funktionale Korrektheit und Rechtzeitigkeit sind unabhängige Kriterien
- Diskussionsannahme Airbag (Zahlen illustrativ, nicht Spezifikation):
 - Crash erkannt (funktional ok)
 - Zündung ausgelöst (funktional ok)
 - Zündung 50 ms zu spät (zeitlich falsch unter angenommener Frist)
- Bei hartem Fristvertrag: Ergebnis nach der Deadline gilt als Anforderungsverletzung
- Nicht pauschal: bei weicher Echtzeit kann ein spätes Ergebnis noch Restnutzen haben

Notizen

Beim Airbag reicht korrekte Logik nicht — unter der Diskussionsannahme einer engen Zündfrist. Der Satz „Ergebnis nach der Deadline ist falsch“ gilt für den harten Fristvertrag, nicht universell für jede Soft-Real-Time-Anzeige. ABS-200 ms auf der Einstiegsfolie ebenfalls als Diskussionsannahme führen.

Lernziele des heutigen Tages

1. Arbeitsdefinition von Echtzeit (Werte- und Zeitbereich) erklären
2. Harte, feste und weiche Echtzeit anhand Aufgabe+Frist unterscheiden
3. Metriken Latenz, Antwortzeit, Jitter und WCET sauber trennen
4. Polling, Interrupts und Foreground/Background zeitlich einordnen
5. Erkennen: Architektur hilft — Deadlines bleiben nachzuweisen

Notizen

Am Ende des Tages: Definition und Klassifikation, Metriken ohne Vermischung, warum lange Super-Loop-Arbeit Fristen gefährdet, und wie Interrupts/RTOS strukturieren — ohne automatischen Determinismus zu behaupten.

Was ist Echtzeit? (Definition)

- Zwei unabhängige Kriterien, beide müssen erfüllt sein:
 1. funktional korrekt — der Wert ist richtig (Wertebereich)
 2. rechtzeitig — der Wert liegt bis zur Frist vor (Zeitbereich)
- Keines der beiden Kriterien folgt aus dem anderen
- Historischer Bezug: „Echtzeitbetrieb“ in Anlehnung an DIN 44300 — Lehrzitat, kein geprüfter Normwortlaut
- Arbeitsdefinition (Lehrkontext): ein System erfüllt Echtzeitanforderungen, wenn spezifizierte Fristen unter den genannten Annahmen eingehalten werden
- Determinismus (Arbeitsbegriff): zeitliches Verhalten unter dokumentierten Bedingungen begrenzt vorhersagbar

Notizen

Die beiden Kriterien sind wirklich unabhängig: Ein Regler kann den perfekten Stellwert viel zu spät liefern, und ein pünktlicher Wert kann fachlich falsch sein. Deshalb prüfen wir beides getrennt. DIN 44300 zitieren wir als historische Traditionslinie, nicht als geprüften Normwortlaut — verbindlich ist hier die Arbeitsdefinition. Determinismus heißt nicht „immer dieselbe Laufzeit“, sondern vorhersagbare Schranken unter benannten Annahmen.

Klassifizierung von Echtzeitsystemen

- Nicht jede Fristverletzung ist gleich zu bewerten
- Unterscheidung nach spezifizierter Konsequenz eines Deadline Miss:
 - Harte Echtzeit (Hard Real-Time)
 - Feste Echtzeit (Firm Real-Time)
 - Weiche Echtzeit (Soft Real-Time)
- Beispiele immer als konkrete Aufgabe + Frist, nicht als ganze Branche dauerhaft einer Klasse

Notizen

In einem Gerät können alle drei Kategorien auf verschiedene Aufgaben verteilt sein. „Automotive = hard“ ist zu grob — Motorregelung und Status-LED sind unterschiedliche Fristverträge.

Harte Echtzeit (Hard Real-Time)

- Spezifikation darf die Frist nicht verletzen
- Deadline Miss = Anforderungsverletzung (unabhängig vom Mittelwert)
- Nutzenkurve (illustratives Modell): Nutzen nach Miss abrupt null oder negativ
- Beispielaufgabe: PWM-Update für Motorstromregelung vor nächstem PWM-Zyklus
- Frist: z. B. $D = 100 \text{ s}$ (Lehrannahme)

i Sicherheit ist möglicher Kontext, nicht Definition

Sicherheitsfolgen für Mensch oder Umwelt können einen harten Fristvertrag begründen — hart bleibt die Klasse aber auch dann, wenn ein Miss „nur“ die Anforderung verletzt.

Notizen

Kein Spielraum im spezifizierten Vertrag: Fristverletzung = Anforderung verletzt. Ob daraus physische Katastrophe folgt, ist Anwendungs- und Safety-Frage — nicht die Definition selbst. Nutzenkurven sind illustrative Modelle.

Feste Echtzeit (Firm Real-Time)

- Ergebnis muss vor einem Übergabepunkt vorliegen
- Danach: Ergebnis für diese Aktion wertlos (Nutzen = 0), ohne zwingend Systemausfall
- Beispielaufgabe: Werkstück sortieren — Kamera klassifiziert, Weiche schleust aus
- Übergabepunkt 150 mm hinter der Kamera, Band 1 m/s $\rightarrow D = 150$ ms (Lehrannahme)
- Miss-Toleranz explizit festlegen, z. B. höchstens 1 verworfenes Teil je 1000 (Lehrannahme)
- Nutzenkurve (illustratives Modell): Sprung auf null nach der Deadline

Notizen

Der Übergabepunkt macht die Frist physikalisch greifbar: $D = s/v = 0,15 \text{ m} / 1 \text{ m/s} = 150 \text{ ms}$.
Späte Prüfung: Teil ist vorbei — dieses Ergebnis verwerfen, die Anlage läuft weiter. Entscheidend für die Spezifikation ist die Zahl bei der Miss-Toleranz: „gelegentlich darf es ausfallen“ ist kein prüfbarer Vertrag, „höchstens 1 von 1000“ schon.

Weiche Echtzeit (Soft Real-Time)

- Deadline-Miss mindert Qualität; Ausfall ist nicht zwingend
- Aktualität / Nutzen nimmt typischerweise ab (kein universelles Video-Nutzenprofil)
- Nutzenkurve (illustratives Modell): allmählicher Abfall — Form applikationsspezifisch
- Beispielaufgabe: Statusanzeige einer Temperatur mit Zielaktualität 1 s
- Fristverletzung: Anzeige veraltet — Qualitätsziel verfehlt, nicht automatisch Systemversagen

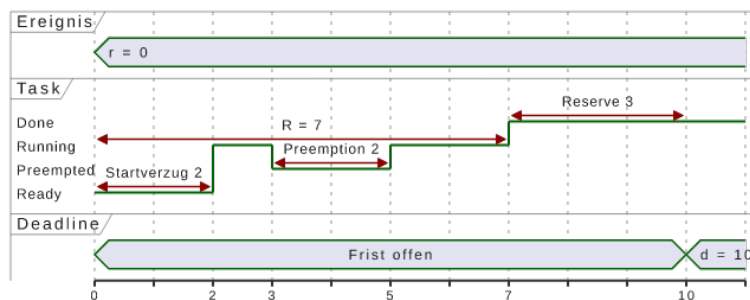
Notizen

Design oft auf Average Case und hohen Durchsatz; gelegentliche Ausreißer werden toleriert. Vermeiden Sie die Erzählung „Video hat immer diese Nutzenkurve“ — die Kurve ist ein illustratives Modell.

Zeitstrahl-Vokabular

Lehrbeispiel (eine Job-Ausführung, Zeiten in ms):

- Ereignis / Release $r = 0$, Start $s = 2$, Fertig $f = 7$, absolute Deadline $d = 10$
- Ausführung 2–3 und 5–7; Präemption 3–5
- Startverzögerung = 2, CPU-Bedarf $C = 3$, Präemption = 2
- Antwortzeit $R = f - r = 7$, Fristreserve = $d - f = 3$
- Relative Deadline $D = d - r = 10$ (daneben notieren)
- Absolute/relative Deadline $\neq$ Periode T

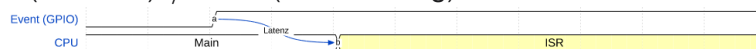


Notizen

C ist CPU-Bedarf (hier 3 ms reine Ausführung), nicht die Antwortzeit. R enthält Warte-/Präemptionszeiten. Periode T ist ein anderer Parameter — Formalisierung in VL2. Absolute Deadline d vs. relative $D = d - r$ klar trennen.

Latenz (Reaktionsverzögerung)

- Latenz: Verzögerung zwischen Ereignis und einer definierten Reaktion
- Interrupt-Latenz: Flanke am Pin bis erste Instruktion der ISR
- Kurze Hardware-Kette: Pin-Flanke $\rightarrow$ IRQ $\rightarrow$ erste Handlerinstruktion $\rightarrow$ HAL-Callback $\rightarrow$ GPIO
- Response Time R (Release $\rightarrow$ Completion): formal vertieft in VL2; $\neq$ Interrupt-Latenz
- Feste Latenz (konstant) $\neq$ Jitter (Schwankung)



Notizen

Zwischen GPIO-Flanke und ISR: Interrupt erkennen, Pipeline leeren, Register sichern (Exception Entry), Sprung in die ISR. Beim Cortex-M oft Größenordnung ca. 12 Taktzyklen Hardware-Latenz — plus Software bis HAL-Callback. R_i kommt formal in VL2.

Jitter

Illustrative Messreihe (Sollperiode $T = 10$ ms):

	t_0	t_1	t_2	t_3	t_4
Ideal	0	10	20	30	40
Ist	0	9,8	20	29,8	40

- Perioden (Ist): 9,8/10,2/9,8/10,2 ms
- Peak-to-Peak-Periodenjitter = $10,2 - 9,8 = 0,4$ ms
- Phasenfehler zu Ideal: $0/ - 0,2/0/ - 0,2/0$ ms; Spanne 0,2 ms (nicht mit Periodenjitter vertauschen)
- Latenzjitter separat: $\max(L) - \min(L)$
- Metrik nennen: Spanne / Peak-to-Peak in ms; Varianz in ms^2 , Standardabweichung in ms
- Regler: Entwurfsfrage (Abtastmodell, Robustheit) — keine pauschale „Stabilitätszerstörung“



Notizen

Periodenjitter und Phasenfehler sind verwandt, aber Zahlen nicht vertauschen: hier PpP 0,4 ms, Phasenspanne 0,2 ms. „Jitter = Varianz“ ist ungenau — Varianz hat Einheit ms^2 . Lernkontrolle: Konstanter Abstand 550 ms statt 500 ms — ist das Jitter? Nein: das ist eine konstante Periodenabweichung (Bias); Jitter beschreibt Schwankung um den Soll-/Mittelwert.

Determinismus und WCET

- WCET: sichere Obergrenze der Ausführungszeit unter Worst-Case-Annahmen
- BCET: untere Grenze / minimale Ausführungszeit
- Deterministisch: unter dokumentierten Bedingungen begrenzt vorhersagbar (nicht: immer identische Laufzeit)
- Trennen: CPU-Bedarf C vs. präemptierte Zeit vs. Antwortzeit R
- Caches, Buskonflikte, Pfade und Interrupts erschweren enge Schranken

Notizen

Durchschnittswerte reichen für harte Echtzeit nicht. Dieselbe Funktion kann kalt/warm, mit/ohne Flash-Waitstates unterschiedlich lang laufen — deshalb Annahmen explizit machen.

Messmaximum vs. WCET

Lehrwerte (erfunden, keine Boardmessung): beobachtetes Maximum 80 s, sichere Obergrenze 120 s.



- Messmaximum $\neq$ WCET: seltenere/ungünstigere Fälle können fehlen
- Annahmen notieren: Cold/Warm-Cache, Flash-Waitstates, Buslast, ausgeführte Pfade
- Vermeiden: „kein D-Cache = immer gleiche Laufzeit“
- Nicht: „WCET absichern nur durch extreme Tests“ — Analyse, Messung und Annahmen gehören zusammen (Vertiefung VL2)

Notizen

Die 80/120 s sind reine Lehrzahlen. Ein Messmaximum aus N Läufen ist eine Stichprobe. WCET-Arbeit kombiniert Messung, Strukturanalyse und dokumentierte Worst-Case-Annahmen — nicht nur „langer Stresstest“.

Zusammenfassung: Metriken

- Deadline d / D : Zeitpunkt bzw. relative Frist bis zum Ergebnis
- WCET: sichere Schranke für den CPU-Bedarf — Messmaximum allein reicht nicht
- Latenz: Verzögerung Ereignis $\rightarrow$ definierte Reaktion (fest $\neq$ schwankend)
- Antwortzeit R : Release bis Completion (inkl. Warte-/Präemptionszeiten)
- Jitter: Schwankungsmaß — Metrik nennen (z. B. Peak-to-Peak in ms)
- Goldene Regel: Vorhersagbarkeit unter Annahmen, nicht Durchsatz

Notizen

Fragen zu diesem Theorieblock? Danach: wie man das klassisch programmiert. Kurz: Mittelwert 1 ms vs. Deadline 10 ms — ohne Schranke kein Nachweis.

Architekturen: Wie programmieren wir das?

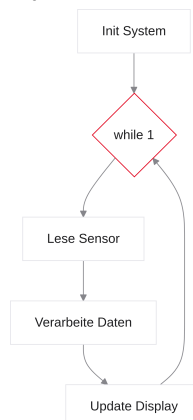
- Wechsel von der Theorie zur Software-Architektur
- Ohne Betriebssystem: Bare-Metal
- Drei klassische Ansätze:
 1. Polling-Loops (Super-Loop)
 2. Interrupt-getriebene Systeme
 3. Foreground/Background-Systeme
- Keiner davon liefert automatisch einen Deadline-Nachweis

Notizen

Code läuft direkt auf der Hardware. Die drei Muster sind historisch und didaktisch der Weg zum RTOS — als Strukturierung, nicht als Garantieautomat.

Architektur 1: Die Polling-Loop (Super-Loop)

- Simplestes Muster: Endlosschleife `while (1)`
- Tasks strikt sequenziell (kooperativ)
- Status wird aktiv abgefragt („Polling“)
- Vorteil: simpel, wenig Overhead, leicht zu debuggen
- Qualifikation: zeitlich begrenzte kooperative Abläufe können Deadlines erfüllen



Notizen

Klassisches Arduino-Muster: linearer Kontrollfluss. Polling heißt: aktiv nachfragen, ob etwas passiert ist. Nicht pauschal „ungeeignet“ — entscheidend ist das Zeitbudget der Runde.

Code-Beispiel: Polling

```
void main(void) {
    System_Init();
    while (1) {
        if (UART_Has_Data()) {    /* Polling */
            Process_Command();
        }
        if (Button_Pressed()) {  /* Polling */
            Toggle_LED();
        }
        Update_Display();
    }
}
```

- Super-Loop-Modell (Lehrannahme): Eingang 0,1 ms, Komm. 0,2 ms, Display 5 ms
- Maximale Runde $\approx 5,3$ ms; bei $D = 10$ ms ggf. ok
- Frage: Was passiert, wenn `Update_Display()` 50 ms braucht?

Notizen

Während einer langen Display-Funktion ist das System für Taster und UART blind. Die Latenz summiert sich über alle Abschnitte der Schleife. Bei 5,3 ms und $D = 10$ ms kann das Modell greifen — bei 50 ms Display ist das Budget gerissen.

Die Gefahr: Das Heavy-Load-Problem

- Eine lang laufende Funktion blockiert die gesamte Schleife
- Auf Eingänge kann nicht reagiert werden
- Fristbudget der Runde wird überschritten
- Typische Sünden: `delay(100)`, blockierende Warteschleifen
- Kurzer Puls zwischen zwei Kontrollen: Verlust hängt von Latch/FIFO/Signalbreite ab

```
void Update_Display(void) {
    /* ~50 ms Ausführungszeit */
    Draw_Pixels();
}
```

Worst-Case-Runde mit Display 50 ms: Budget $D = 10$ ms gerissen

Notizen

Eine 50-ms-Funktion lässt die Super-Loop stehen. Ob ein kurzer Tasterimpuls verloren geht, hängt davon ab, ob die Hardware latched/FIFOed oder nur während der Abfrage sichtbar ist — nicht pauschal „immer verloren“.

Folie

26 / 58

Fazit: Polling

- Vorteile: simpel, keine ISR-Races, feste Reihenfolge
- Nachteile: Priorisierung fehlt, lange Abschnitte gefährden Fristen, oft höherer Strom (aktives Warten)
- Einsatz: passende Zeitbudgets und wenige asynchrone Ereignisse
- Nicht: „Polling ist grundsätzlich ungeeignet für Echtzeit“

Notizen

Für einen einfachen Temperatursensor mit großzügigem D oft ok. Sobald mehrere asynchrone Ereignisse mit engen Fristen kommen, wird das Modell unpraktisch — dann Interrupts oder RTOS als Strukturierung.

Folie

27 / 58

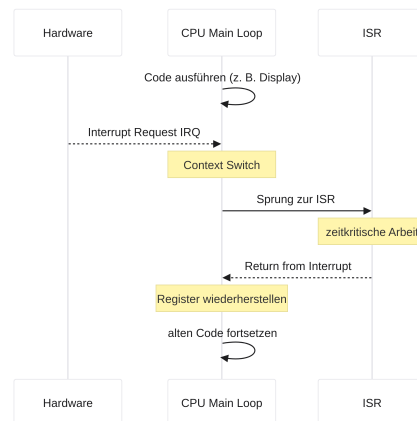
Architektur 2: Interrupts zur Rettung

- Interrupt: Hardware-Signal, das die CPU unterbricht
- Kontext sichern, Sprung in die ISR
- Nicht mehr fragen — die Hardware meldet sich
- Ereignisse priorisierbar (z. B. Not-Halt vor Display)
- Interrupts erzeugen nicht automatisch Determinismus oder Fristnachweise

Notizen

Asynchrone Reaktion mit geringer Latenz, unabhängig von der Main-Loop — gegen das Heavy-Load-Problem. Höhere IRQs, Maskierung und lange ISRs bleiben Timing-Faktoren.

Der Ablauf eines Interrupts



Notizen

Context Switch sichert Register und Program Counter. Danach ISR, dann Rückkehr genau an die Unterbrechungsstelle. Details Cortex-M: Deep Dive gleich im Anschluss.

Code-Beispiel: Timer-Interrupt (STM32 HAL)

Setup in `main()`:

```
HAL_TIM_Base_Start_IT(&htim2);

while (1) {
    Update_Display(); /* Hintergrund */
}
```

Callback (ISR):

```
void HAL_TIM_PeriodElapsedCallback(
    TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        Toggle_LED(); /* zeitkritisch */
    }
}
```

Notizen

`_IT` startet den Timer im Interrupt-Modus. Die LED kann periodisch toggeln, auch wenn `Update_Display()` lange braucht — das ist Strukturierung, kein vollständiger harter Fristnachweis.

Folie

30/58

Gefahren von Interrupts

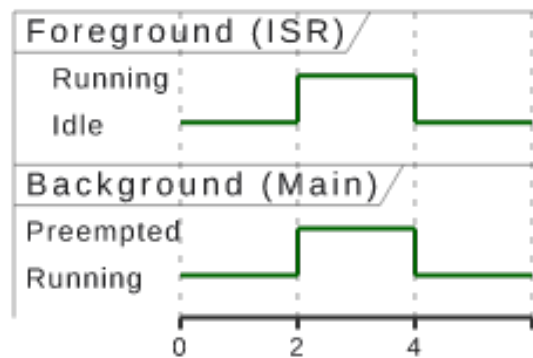
- Race Conditions bei gemeinsamen globalen Variablen
- Lange ISRs blockieren niederpriorige IRQs und die Main-Loop
- Budget je ISR festlegen: kurz halten, kein `printf`, keine Delays, Maskierungsdauer begrenzen
- Jitter: laufende höhere IRQs verzögern niederpriorige ISRs
- Flag-Race (Lehrfall): Main liest Flag → neues Event → Main löscht Flag — `volatile` allein reicht nicht

Notizen

Asynchronität bringt Korrektheitsfallen. In der ISR: Auftrag/Flag setzen, sofort zurück. Lange Arbeit gehört in den Hintergrund oder später in Tasks. „Kurz halten“ ist als Budget zu formulieren: eine Zahl pro ISR, die in die Antwortzeitrechnung der niederpriorigen Arbeit eingeht — inklusive der Zeit mit gesperrten Interrupts. Beim Flag-Race braucht es atomare Abschnitte oder sichere Übergabeprotokolle; `volatile` verhindert nur Compiler-Optimierungen und liefert weder Atomarität noch Schutz bei Read-Modify-Write — Vertiefung bei Synchronisation.

Architektur 3: Foreground / Background

- Kombination: Main-Loop + Interrupts
- Background: unkritische Arbeit (`main`)
- Foreground: zeitkritische ISRs
- ISR setzt Auftrag; lange Main → Antwortzeit kann groß werden
- Standard für kleine bis mittlere Bare-Metal-Projekte

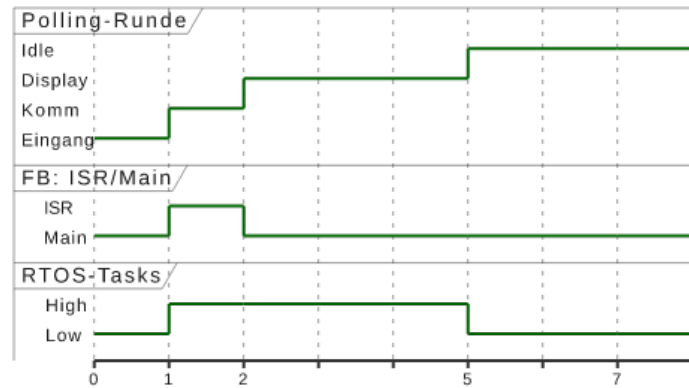


Notizen

ISR präemptiert die Main-Loop. Fertig → Main läuft weiter. Das ist das klassische Bare-Metal-Muster vor dem RTOS. Wenn Main den Auftrag erst nach langer Arbeit bearbeitet, wächst R trotz schneller ISR.

Drei Architektur-Zeitachsen

Lehrskizze derselben Frist $D = 10$ ms:



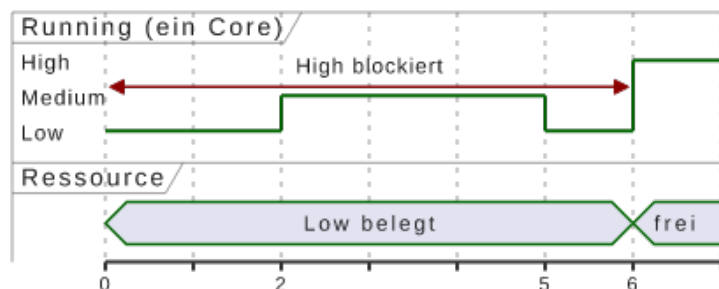
- Polling: Reaktion erst in der nächsten Abfrage der Runde
- Foreground/Background: ISR reagiert sofort, Arbeit oft erst in Main
- RTOS: Scheduler wählt Ready-Tasks — Fristen trotzdem nachweisen

Notizen

Die drei Achsen sind didaktische Vereinfachungen. Polling kann bei kleinem Rundentakt Fristen halten; FB verschiebt oft Arbeit in die Main; RTOS strukturiert Prioritäten — ersetzt aber keine Response-Time-Analyse (VL2).

Grenzen der klassischen Architekturen

- Warum reicht Foreground/Background oft nicht mehr?
- Spaghetti-Falle: ISRs setzen Flags, Main pollt endlos
- Bei vielen Tasks verliert man den Überblick über den Kontrollfluss
- Priorisierung und Schlafen von Tasks werden nötig
- Teaser Prioritätsinversion (Task-Kontext): Low hält Ressource, High blockiert, Medium verzögert Low — nur eine Running-Spur; kein Mutex in ISR → VL4



Notizen

ISR darf nicht lange rechnen → Flag. Main wird Flag-Suppe. Deadlines in der Main-Loop sind dann kaum noch garantierbar. Die eine Running-Spur zeigt das PI-Muster ohne parallele „Running“-Illusion — Details und Priority Inheritance in VL4.

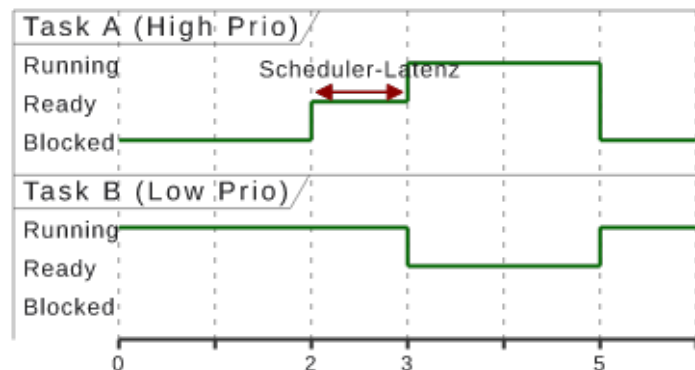
Der Paradigmenwechsel

- Wenn Bare-Metal nicht skaliert: RTOS (Real-Time Operating System)
- RTOS: Strukturierungswerkzeug (Tasks, Prioritäten, Blockieren) — kein automatischer Deadline-Nachweis
- Bare-Metal kann ereignisgetrieben schlafen (WFE/Idle) — nicht nur aktives Polling
- Monolith → unabhängige Tasks unter Scheduler-Kontrolle

Notizen

Ab hier ändert sich die Denkweise: nicht mehr eine riesige `while (1)` orchestrieren, sondern Tasks und Scheduler. FreeRTOS folgt in späteren Terminen. Vermeiden Sie die Erzählung „Interrupts + RTOS = Determinismus“.

Vorschau: Das Task-Modell



- Task B rechnet; Event weckt Task A
- RTOS entzieht Task B die CPU (Präemption) und gibt sie Task A
- Pro Core ist immer genau eine Task **Running** — **Ready** heißt wartend, nicht rechnend
- Ob Deadlines gehalten werden, bleibt Analyse-/Messaufgabe (VL2)

Notizen

Echtes präemptives Multitasking auf einem Core. Achten Sie auf die Zustände: Zwischen **Ready** bei 2 und **Running** bei 3 liegt die Scheduler-Latenz — das Aufwecken kostet Zeit und ist nicht null. Verbreiteter Zeichenfehler in Übungen: zwei Tasks gleichzeitig **Running** auf einem Core; das kann es nicht geben. Details zu Scheduler-Latenzen, Mutexen (nie in einer ISR) und Prioritätsinversion folgen mit FreeRTOS in VL4.

Deep Dive: Die ARM Cortex-M Architektur

- Um Interrupts zu beherrschen, müssen wir die CPU verstehen
- Der STM32H755: Cortex-M7 und Cortex-M4 (AMP)
- Cortex-M ist für eingebettete Echtzeitsysteme optimiert
- Merkmale:
 - 32-Bit-**Programmier-/Registermodell** (nicht jede Busbreite ist „32 Bit“)
 - Deterministisches Interrupt-Verhalten (unter dokumentierten Annahmen)
 - Keine MMU, sondern MPU (Memory Protection Unit)

Notizen

Cortex-M („Microcontroller“) ist auf Determinismus ausgelegt — anders als Cortex-A mit MMU. „32-Bit“ bezieht sich auf Register/ALU/Programmierung. Quellen: PM0214 (Cortex-M4 Programming Manual), RM0399.

Das Register-Set der CPU

Register	Rolle
R0–R12	Allgemeine Datenregister
R13 (SP)	Stack Pointer
R14 (LR)	Link Register
R15 (PC)	Program Counter
xPSR	Statusregister (Flags)

- 16 Basis-Register à 32 Bit
- Bei Exception Entry: **nicht** alle 16 automatisch auf den Stack

Notizen

Die ISR nutzt dieselben Register. Hardware sichert nur einen Teil — der Rest ggf. durch Compiler/Software. Details auf der nächsten Folie.

Exception Entry (nicht „voller Taskwechsel“)

- Interrupt unterbricht den laufenden Thread
- Hardwaregesteuerter **einfacher** Stackframe (ohne FPU):

Offset	Inhalt
0...7	R0, R1, R2, R3, R12, LR, PC, xPSR

- Grundgröße: $8 \times 4 = 32$ Byte (plus ggf. Alignment)
- R4–R11: bei Bedarf durch Compiler/Software (nicht automatisch)
- LR im Handler enthält typisch **EXC_RETURN** — kein gewöhnlicher Funktionsrückprung

Notizen

Lehrannahme: einfaches Stacking ohne FP-Frame, idealisierte Speicherbedingungen. Exception Entry $\neq$ RTOS-Taskwechsel: dieselbe unterbrochene Arbeit kann fortgesetzt werden, ohne einen anderen Task zu wählen. Quelle: PM0214 / Arm Exception Model.

Von der Pin-Flanke zum HAL-Callback



- Hardware-Entry und HAL-Software farblich/logisch trennen
- Idealisiert oft ≈ 12 Entry-Zyklen (Cortex-M4, passende Annahmen) — **keine** Flanke-bis-Callback-Garantie

Notizen

Jede Latenzangabe braucht klaren Anfang und Endpunkt. „GPIO-Flanke → HAL-Callback“ enthält Peripherie, NVIC, Stacking, Vektorabruf, C-Prolog und HAL. Quelle: Arm Interrupt-Latency-Guide (Joseph Yiu) — Voraussetzungen beachten.

Der NVIC: Herr der Interrupts

- NVIC = Nested Vectored Interrupt Controller
- Sitzt im Cortex-M-Kern, eng mit der CPU verzahnt
- **Architekturgrenze** Cortex-M: bis zu 240 externe IRQs
- **STM32H755-Implementierung:** bis zu ca. 150 maskierbare Kanäle, 16 Prioritätsstufen — **pro Kern eigener NVIC**
- Vectored: direkter Sprung über Vector Table

Notizen

Nicht Architekturgrenze und Chip-Implementierung vermischen. Beide Kerne (M7/M4) haben eigene NVIC-Instanzen. Datenblatt STM32H755 / RM0399.

Prioritäten im NVIC

- Nested: höhere Dringlichkeit darf niedrigere ISR unterbrechen
- Falle: **kleinere Zahl = höhere Priorität**
- Priorität 0: höchste *konfigurierbare* Priorität
- Feste Systemausnahmen (z. B. NMI) gesondert einordnen
- Zustände getrennt denken: Enabled / Pending / Active; Peripherieflag $\neq$ NVIC-Pending

Notizen

Beispiel (Lehr): TIM2 mit Preemption-Prio 5 darf UART mit Prio 8 unterbrechen, sofern keine Maskierung. Priorität allein erzeugt keine obere Latenzgrenze — Maskierung und höhere ISRs zählen.

Folie

42 / 58

Preemption Priority vs. Subpriority

- Prioritäts-Bits lassen sich aufteilen (Grouping)
- Preemption Priority: darf ein IRQ einen laufenden IRQ unterbrechen?
- Subpriority: bei gleichzeitigem Warten — wer zuerst?
- Lehrbeispiel 2+2 Bits: $A=(1, 0)$, $B=(1, 3)$ — A zuerst bei Pending; A präemptiert laufendes B **nicht** (gleiche Preemption Priority)

Notizen

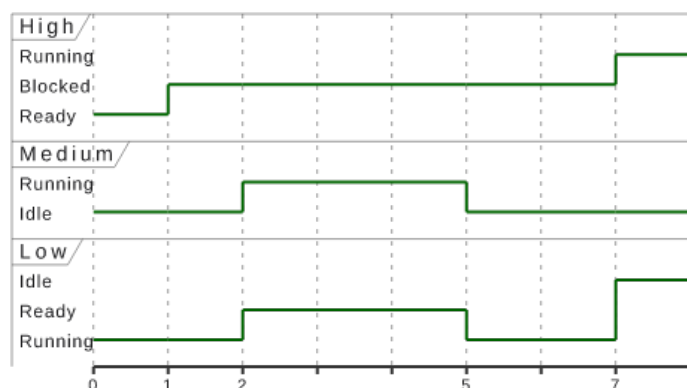
CMSIS/HAL-Werte und rohe Registerbits nicht vermischen; `__NVIC_PRIO_BITS` und Grouping aus dem Projekt prüfen. Keine ungesicherte Bitverschiebung in Anwendungscode.

Folie

43 / 58

Prioritätsinversion (Teaser)

- **Task-Kontext** (nicht ISR): Low hält Ressource, High wartet, Medium verzögert Low
- Pro Kern maximal eine Spur gleichzeitig *Running*
- Mutex-Erwerb/-Freigabe markieren; **kein** blockierender Mutex in einer ISR
- Vollständige Behandlung (Priority Inheritance): VL4



- Teaser: Mars Pathfinder (1997) — Details in VL4

Notizen

Bei $t \approx 1$: High will Ressource, Low hält sie → High Blocked. Medium präemptiert Low → High bleibt

Tail-Chaining: ARM's Geheimwaffe

- Zwei Interrupts A und B treten kurz nacheinander auf
- Ohne Optimierung: Entry A → A → Return A → Entry B → B → Return B
- Mit Tail-Chaining: Entry A → A → Übergang zu anstehendem zulässigem B → B → Return
- Idealisiertes Cortex-M4-Modell: Return+neuer Entry $\approx 12 + 12$ vs. Übergang ≈ 6 Zyklen
→ **18 Übergangszyklen gespart** (nicht „jede Latenz = 6“)

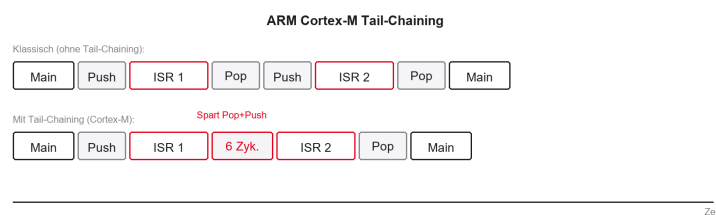


Figure 1: ARM Cortex-M Tail-Chaining: Vergleich mit und ohne Overhead

- Bedingung: B steht beim Exception Return an **und** darf angenommen werden

Notizen

Voraussetzungen: passende Speicherbedingungen, keine Maskierung, kein FP-Kontext-Wechsel. Kein Tail-Chaining zwischen zwei Cores. Late Arrival und nested Preemption sind abgegrenzte Alternativen. Quelle: Arm Interrupt Latency Blog / Cortex-M TRM.

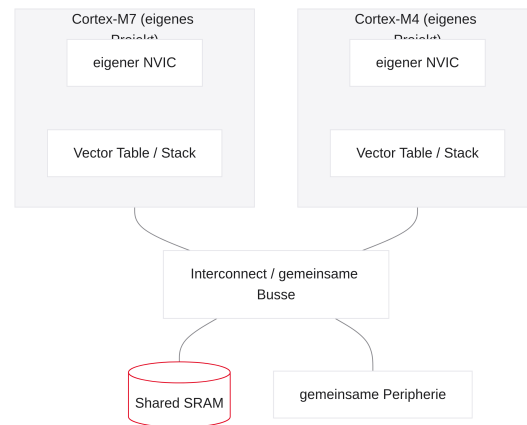
Exkurs Hardware: Der STM32H755 / NUCLEO-H755ZI-Q

- **Board:** NUCLEO-H755ZI-Q (Nucleo-144, MB1363, internes SMPS – Q)
- **MCU auf dem Board:** STM32H755ZIT6 (LQFP144, 2 MB Flash) — nicht mit dem Boardnamen verwechseln
- Cortex-M7 (Spec bis 480 MHz): Leistung, Caches, FPU
- Cortex-M4 (Spec bis 240 MHz): oft für Regelung genutzt
- Beide teilen Chip, Busse und teilweise Speicher → **Interferenz möglich**

Notizen

LEDs MB1363 (UM2408 §7.6): LD1 grün PB0 oder PA5 (Jumper SB65/SB54); LD2 gelb **PE1** (fest); LD3 rot **PB14**. HIGH = an. Keine .ioc im Kursrepo — Konfiguration in CubeIDE am gestellten Board. Quellen:

Warum Dual-Core (AMP) in der Echtzeit?



Lehrbeispiel-Verteilung:

- M7: Ethernet, GUI, Filter
- Latenz kann durch Caches schwanken

Lehrbeispiel-Verteilung:

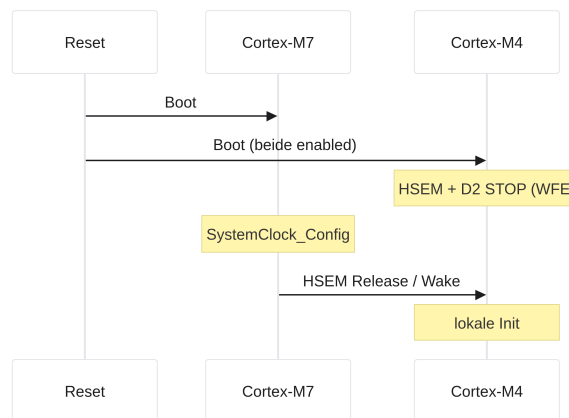
- M4: Motor-PWM, Timing-Loops
- oft besser vorhersagbar (kein D-Cache)
- **Keine Garantie** „M4 = hard, M7 = soft“ — beide können Echtzeitaufgaben tragen
- Konflikt um Shared SRAM/Peripherie → Zuständigkeit + Synchronisation (HSEM)
- HSEM: Zugriff/Benachrichtigung — **keine** Cache-Kohärenz

Notizen

AMP hilft, Welten zu trennen, ersetzt aber keinen Timing-Nachweis. Gemeinsame Ressourcen bleiben. Inter-Core-Datenübergabe und M7-Cache: offene Entwurfsaufgabe späterer Termine.

Gewählte Bootfolge (CubeIDE-Typ)

- Beschreibt die **häufige CubeIDE-/Template-Sequenz** bei beiden aktivierten Kernen — keine universelle Reset-Regel
- Typisch: M4 startet, meldet sich per HSEM und geht in **D2 STOP (WFE)**; M7 konfiguriert Clocks und weckt M4 (HSEM-Release)
- Das ist **Software-Sync nach gemeinsamem Boot**, kein „M4 ab Reset festgehalten“
- Echtes Hold-from-Reset nur über Option Bytes (z. B. BCM4/BCM7) — nicht beiläufig ändern



Notizen

Option Bytes und Images sind Voraussetzungen. Andere Bootkonfigurationen: AN5361, AN5557. Cube-Template ≠ BCM4-gated Hold. Keine beiläufige Option-Byte-Änderung als Standardschritt.

Einführung in die Tools: STM32CubeIDE

- STM32CubeIDE: Eclipse-basierte C/C++-IDE für STM32
- Device Configuration Tool (ehemals CubeMX)
- Grafische Pin-Zuweisung und Clock-Tree
- Codegenerator für die HAL

Notizen

Statt hundert Seiten Register-Doku: Pin wählen, Funktion zuweisen, Takt konfigurieren — der Generator liefert die Basis-Initialisierung.

Das HAL-Konzept

- Nicht direkt in Register tippen (Standardfall)
- HAL: lesbar und oft portabel innerhalb der STM32-Familie
- Nachteil: leichter Aufruf-Overhead
- Extreme Echtzeit: LL oder direkte Register
- Vergleich nur bei **gleichwertiger** Initialisierung/Statusbehandlung

```
/* HAL (nach vollständiger Init) */
HAL_TIM_Base_Start_IT(&htim2);

/* LL: Enable allein ≠ komplette Init */
LL_TIM_EnableIT_UPDATE(TIM2);
LL_TIM_EnableCounter(TIM2);
```

Notizen

Nur Enable-Bits setzen ersetzt keine CubeMX-Init, Clock-Enable und Statusflags. Für den Großteil der Vorlesung reicht HAL.

Projektstruktur und Code-Generierung

- Pro Core eigener Ordner (CM4 / CM7)
- Je eigene `main.c` und `stm32h7xx_it.c`
- Eigener Code nur zwischen `USER CODE BEGIN` und `USER CODE END`
- Code außerhalb dieser Blöcke wird beim erneuten Generieren gelöscht
- Keine zweite `while(1)` in USER-Blöcken schachteln; Callbacks nicht doppelt definieren

Notizen

Die wichtigste Anfängerregel: nur in USER-CODE-Blöcken schreiben — sonst ist die Arbeit mit einem Speichern in der GUI weg.

Laboraufgabe 1: Hello World (LED Blink)

- Ziel: Toolchain prüfen, Cortex-M4 nutzen
- Schritte:
 1. Projekt anlegen (**Board** NUCLEO-H755ZI-Q / MCU STM32H755ZIT6)
 2. LED **PE1** (LD2 gelb auf MB1363) als GPIO_Output für CM4
 3. Code generieren; GPIO-/Clock-Init prüfen
 4. In `main.c` (CM4) toggeln

```
/* USER CODE BEGIN 3 - Inhalt der vorhandenen while(1) */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
HAL_Delay(500); /* blockiert; kompletter LED-Zyklus ≈ 1 s */
/* USER CODE END 3 */
```

Notizen

Toggle alle 500 ms → zwei Flanken/s, aber ein voller On-Off-Zyklus ≈ 1 Hz. Wenn die LED blinkt, laufen IDE, Flash und Debugger.

Laboraufgabe 2: Das Polling-Desaster

- Ziel: Grenzen der Super-Loop spüren
- `Heavy_Load()` so schreiben, dass der Compiler die Arbeit nicht entfernt (Ergebnis nutzen)
- **Konstante** Last verlängert den Abstand konstant (noch kein Jitter); für Jitter bewusst **variable** Lasten

```
/* in vorhandener while(1), USER CODE BEGIN 3 */
Heavy_Load(800000); /* Iterationen ≠ bekannte ms-Dauer */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
HAL_Delay(500);
```

- Beobachtung: Warum weicht der sichtbare Takt vom Soll ab?

Notizen

Buildoptionen dokumentieren. `Heavy_Load`-Iterationen sind keine bekannte Zeitdauer. Konstante Verlängerung ≠ Periodenjitter; für Schwankungen Last wechseln.

Laboraufgabe 3: Rettung durch den Timer

- Ziel: periodische LED-Flanken per TIM2-IRQ
- In der `.ioc` (CM4): TIM2, Clock Source Internal; NVIC „TIM2 global interrupt“
- Vor der Schleife: `HAL_TIM_Base_Start_IT(&htim2)` mit Fehlerprüfung



- Upcounter-Modell:

$$f_{\text{UPD}} = \frac{f_{\text{TIM}}}{(PSC + 1)(ARR + 1)}$$

- Lehrbeispiel (nur wenn $f_{\text{TIM}} = 100 \text{ MHz}$ in der `.ioc` gilt): $PSC = 9999$, $ARR = 4999 \rightarrow 2 \text{ Hz Update (500 ms)}$, LED-Zyklus $\approx 1 \text{ Hz}$
- f_{TIM} aus Clock-Tree (APB-Prescaler/TIMPRE: oft $2 \times \text{PCLK}$ bei Prescaler $\neq 1$) — nicht automatisch `SystemCoreClock`
- TIM2 ohne Repetition Counter: $(RCR + 1) = 1$

Notizen

Ohne NVIC-Haken und Handler-Kette (`TIM2_IRQHandler` → `HAL_TIM_IRQHandler` → `Callback`) feuert nichts Sichtbares. TIM2-Kernel-Takt in CubeMX ablesen. Updatefrequenz $\neq$ Blinkzyklus (Toggle → halbe Frequenz für On-Off).

Laboraufgabe 3: Der Callback-Code

- Toggle/`HAL_Delay` aus der Main-Schleife entfernen — dort z. B. nur noch `Heavy_Load()`
- Callback **einmal** im vorgesehenen USER-Block (nicht doppelt definieren):

```

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); /* LD2 gelb */
    }
}
/* USER CODE END 4 */

```

- Messbare Aussage: unter dieser Konfiguration periodische Flanken trotz Hintergrundlast — **kein** automatischer harter Fristnachweis

Notizen

Foreground/Background: ISR toggelt, Main rechnet. „Unabhängig von CPU-Last / Jitter ≈ 0 “ durch Messung ersetzen (Aufgabe 4).

Folie

55 / 58

Laboraufgabe 4 (Bonus): Drei Oszi-Messungen

Messung	Was	Aussage
A	Puls um <code>Heavy_Load()</code>	verstrichene Dauer inkl. möglicher IRQs
B	periodische Flanken Main vs. TIM2-Callback	gleiche Sollperiode; konstante vs. variable Last getrennt
C	Stimulus/Referenz vs. früher Handler-Marker	Anfang/Ende nennen; HAL-Callback $\neq$ reine Entry-Zeit

- Pin z. B. PA0 erst nach Pinbelegung prüfen; Trigger, Zeitbasis, Sampleanzahl notieren
- Schema: Min/Max/Mittel/Peak-to-Peak — Zahlen **erst nach Messung** eintragen
- Ersetze „Jitter ≈ 0 “ durch „gemessene Schwankung unter dieser Konfiguration innerhalb ...“

Notizen

Referenz und Ereignis brauchen gemeinsamen Zeitbezug (kein Vergleich zweier unabhängiger Signale). Optimierung, höhere IRQs und Debug-Halt beeinflussen Ergebnisse. Optisch stabiles Blinken ist kein harter Fristnachweis.

Folie

56 / 58

Zusammenfassung des Tages

- Echtzeit = Rechtzeitigkeit (Fristvertrag), nicht pure Rechenleistung
- Hard/Firm/Soft: Konsequenz und Nutzen nach Miss spezifizieren
- Messmaximum $\neq$ sichere WCET-Schranke; Jitter $\neq$ Varianz (Metrik nennen)
- Polling kann Deadlines erfüllen — lange unzerlegte Arbeit oft nicht
- Interrupts/RTOS helfen strukturieren; Deadlines weiterhin nachweisen
- Exception Entry $\neq$ Taskwechsel; NVIC/Tail-Chaining mit Annahmen
- Labor: NUCLEO-H755ZI-Q, TIM2-Pfad messen statt „Jitter ≈ 0 “ behaupten

Notizen

Kurzkontrolle: Erfüllt Mittelwert 1 ms die Deadline 10 ms? (Nein allein.) Ist konstanter Abstand 550 ms Jitter? (Zuerst Periodenabweichung.) Toggle alle 500 ms → LED-Zyklus ≈ 1 Hz. Separater M4 garantiert nicht alle Fristen.

Folie

57 / 58

Ausblick auf Termin 2

- Bare-Metal mit vielen IRQs skaliert nicht ewig (Spaghetti-Flags)
- Nächste Woche: Scheduling-Theorie (Task-Modell, Utilization, RMS/EDF)
- Praxis: eigener einfacher Round-Robin-Scheduler auf dem STM32
- Vorbereitung: heutiges Projekt mitbringen

Notizen

Danach der Weg zu FreeRTOS. Bitte Projekte speichern — wir bauen darauf auf.

Folie

58 / 58

Quellenhinweise (Hardware)

- ST, STM32H755-Datenblatt / RM0399 (NVIC ≈ 150 Kanäle/Kern, Busse, Timer)
- ST, UM2408 (NUCLEO-H7 MB1363, LED/Pinbelegung)
- ST, AN5361 / AN5557 (Dual-Core CubeIDE bzw. Architektur/Boot)
- ST, PM0214 (Cortex-M4 Exceptions / Stackframe)
- Arm CMSIS-Core NVIC; Arm Interrupt-Latency-Guide (Voraussetzungen idealer Zykluszahlen ≈ 12 Entry / 6 Tail-Chaining)
- Kopetz / Buttazzo: Begriffe und Scheduling (Ausgabe im Semesterapparat)

Notizen

Dokumentrevisionen am Board/IDE-Stand prüfen. Keine Messwerte aus Lehrbeispielen als Board-Messungen ausgeben.